SOCIAL EAGLE
learn. build. operate

CAIE Course Program

Assignment 5 - LangChain | learn. build. operate

1. What you will build

Build a small Python program that uses the **LangChain** framework to send an input to an **OpenAI** model and print the model's output. You will then connect **LangSmith** so that every run is traced and you can see the input, the output, and timing in the LangSmith dashboard. You will set up a clean Python environment, install the libraries, add your API keys, run the program, and confirm the run appears in LangSmith.

Difficulty: Beginner to Medium · Tools: Python, LangChain, OpenAI, LangSmith

2. What your app must do

- Use the **LangChain** framework to talk to the model (do not call the OpenAI API directly).
- Send an input or prompt to an OpenAI chat model and print the response in the terminal.
- Enable **LangSmith** tracing so the run is logged and visible in your LangSmith project.
- Read all API keys from environment variables - never hard-code or commit them.
- Keep all your code in a single file named `main.py`.

> **Keep your keys safe**
> API keys are secrets. Store them in environment variables (or a .env file that you add to .gitignore) and make sure they are never pushed to GitHub. Think about what your program should do if a key is missing.

3. Step-by-step setup

Follow these steps in order. Commands are shown for both macOS/Linux and Windows where they differ.

Step 1 - Create a project folder

```
mkdir langchain-app
cd langchain-app
```

Step 2 - Create a virtual environment (venv)

A virtual environment keeps this project isolated from the rest of your system.

```
python -m venv venv
# (on some systems use 'python3' instead of 'python')
```

Step 3 - Activate the virtual environment

```
# macOS / Linux:
source venv/bin/activate

# Windows (PowerShell):
venv\Scripts\Activate.ps1

# Windows (Command Prompt):
venv\Scripts\activate.bat
```

When activated, you will see `(venv)` at the start of your terminal prompt.

Step 4 - Install the libraries

Install LangChain, the OpenAI integration, and the LangSmith client inside the activated venv.

```
pip install langchain langchain-openai langsmith
```

Step 5 - Add your API keys and turn on tracing

Get an OpenAI key from platform.openai.com and a LangSmith key from smith.langchain.com, then set them as environment variables before running. (Variable names follow the current LangSmith quickstart.)

```
# macOS / Linux:
export OPENAI_API_KEY="your-openai-key"
export LANGCHAIN_API_KEY="your-langsmith-key"
export LANGCHAIN_TRACING_V2="true"
export LANGCHAIN_PROJECT="langchain-assignment"

# Windows (PowerShell):
$env:OPENAI_API_KEY="your-openai-key"
$env:LANGCHAIN_API_KEY="your-langsmith-key"
$env:LANGCHAIN_TRACING_V2="true"
$env:LANGCHAIN_PROJECT="langchain-assignment"
```

Step 6 - Run your program

```
python file_name.py
```

You should see the model's response in the terminal, and the run should appear in your project on the LangSmith dashboard.

Step 7 - Deactivate when finished

```
deactivate
```

4. How it should behave

Your wording can differ, but a single run should look roughly like this:

```
Input: Tell me a fun fact about Tamil Nadu.

Output: <the model's response prints here>

(the same run also shows up in LangSmith)
```

5. What to submit

- Push your code to **GitHub**.
- Share your GitHub repository link in **WhatsApp**.
- Post your screen-recorded video of the app working (including the LangSmith trace) in the **community**.

Reminder: build the app yourself - the steps above are your setup roadmap, not the answer. Keep your API keys out of GitHub. Good luck!